Mochilas por fuerza bruta.

De elegir artículos a asignar armas con restricciones.
Una hora para entender, programar y comprobar.

C++ | Selección binaria | Po · LAB1 2024-1

Material de apoyo · Presentación interactiva sin conexión
Flechas: avanzar · F: pantalla completa · Guion docente en la carpeta «Para grabar»

Antes del primer `for`

Tres preguntas determinan cómo será la solución.

DECISIÓN

¿Qué elijo?

Un artículo entra o no entra.
Un arma recibe un destino.

RESTRICCIONES

¿Qué debe cumplirse?

Capacidad, exclusividad,
tipo y prerrequisitos.

OBJETIVO

¿Cuándo termino?

Conservar el mejor peso.
O encontrar alguna solución.

> “Cabe” demuestra factibilidad. Para decir “es la mejor”, necesitas comparar o alcanzar una cota superior.

Una mochila. Cuatro decisiones.

Cada artículo se usa como máximo una vez. Maximiza el peso sin exceder 15.

ARTÍCULO 1	ARTÍCULO 2	ARTÍCULO 3	ARTÍCULO 4
1	2	10	4

TODOS LOS ARTÍCULOS

1 + 2 + 10 + 4 = 17

No caben.

UNA SELECCIÓN ÓPTIMA

1 + 10 + 4 = 15

Alcanza la capacidad.

En este ejercicio no hay valores o ganancias distintos del peso.

Un número genera una selección.

El primer artículo está a la izquierda. El último dígito cambia más rápido.

Código decimal **11** Capacidad 15

ARTÍCULO 1	ARTÍCULO 2	ARTÍCULO 3	ARTÍCULO 4
1	**2**	**10**	**4**
1 · entra	**0 · fuera**	**1 · entra**	**1 · entra**

Peso 15 ≤ capacidad 15
Selección factible. Espacio libre: 0

4 artículos → 2^4 = 16 candidatos
Códigos 0 a 15. El código identifica la selección; no es su peso.

Código: **1 × 8 + 0 × 4 + 1 × 2 + 1 × 1 = 11**
Cromosoma: 1 0 1 1 · Suma de pesos: 1 + 10 + 4 = 15

Genera. Evalúa. Conserva.

El candidato cambia en cada vuelta. La mejor selección debe guardarse.

1. Recorre códigos de 0 a $2^n - 1$.
2. Convierte el código a n dígitos binarios.
3. Suma los pesos elegidos.
4. Si cabe y mejora, copia la selección.

Código	Selección	Peso	¿Cabe?
0	`0000`	0	Sí
3	`0011`	14	Sí
11	`1011`	15	Sí
15	`1111`	17	No

> El programa recorre todos los candidatos. La tabla muestra solo cuatro para explicar el proceso.

¿Llevar el más pesado primero?

Capacidad 6. Artículos de pesos 4, 3 y 3.

ELECCIÓN VORAZ

4 + …

Quedan 2 libres.
No entra ninguno de los artículos de peso 3.

Factible · Peso 4

MEJOR SELECCIÓN

3 + 3 = 6

Se usan dos artículos distintos.
Cada uno se elige una vez.

Óptima · Peso 6

Este contraejemplo demuestra que esa regla voraz no garantiza el máximo peso.

La condición que conserva la respuesta.

```
for (int codigo = 0; codigo < total; codigo++) {
    cargarCromosoma(codigo, n, seleccion);
    long long peso = 0;
    for (int i = 0; i < n; i++)
        if (seleccion[i]) peso += pesos[i];

    if (peso <= capacidad && peso > mejorPeso) {
        mejorPeso = peso;
        for (int i = 0; i < n; i++)
            mejor[i] = seleccion[i];
    }
}
```

`peso <= capacidad`

La igualdad sí cabe.

`peso > mejorPeso`

Solo reemplaza si mejora.

Copia el arreglo

El siguiente candidato cambiará `seleccion`.

Código completo: mochila_binaria.cpp
Coste: $O(n \cdot 2^n)$.

Una respuesta correcta puede dejar espacio.

Predice la salida antes de ejecutar cada caso.

Pesos	Capacidad	Selección	Total	Libre
1, 2, 10, 4	15	`1 0 1 1`	15	0
1, 2, 10, 4	9	`1 1 0 1`	7	2
4, 3, 3	6	`0 1 1`	6	0
1, 2, 10, 4	0	`0 0 0 0`	0	0

> Con capacidad 9, el peso 10 no cabe y todos los demás juntos suman 7. No puedes llenar las 2 unidades restantes.

Si hay varias selecciones con el mismo mejor peso, cualquiera sirve salvo que el enunciado pida un desempate.

Un artículo, un destino.

Con dos mochilas, cada artículo tiene tres opciones.

0

Sin usar

El artículo queda fuera.

1

Mochila 1

Se asigna a M1.

2

Mochila 2

Se asigna a M2.

> $(G + 1)^A$ candidatos para G mochilas y A artículos. El +1 corresponde a no usar el artículo.

Un único dígito por artículo impide asignarlo a dos mochilas a la vez.

Contar en base 3.

Tres artículos: A, B y C. Dos mochilas. $3^3 = 27$ asignaciones.

Código decimal **15**

ARTÍCULO A	ARTÍCULO B	ARTÍCULO C
1	2	0
Mochila 1	Mochila 2	Sin usar

1 2 0 en base 3 → 1 × 9 + 2 × 3 + 0 = 15

Después de 0 0 2 viene 0 1 0. El contador vuelve a cero después de 2 2 2.

Aquí solo representamos destinos. La validez dependerá de las restricciones del problema.

Po necesita una mochila por guerrero.

Encuentra alguna asignación que permita vencer a todos.

1. Cada arma se usa a lo sumo una vez.
2. El tipo debe ser permitido por el guerrero.
3. Los requisitos van en la misma mochila.
4. El poder total supera al del guerrero.

EL OBJETIVO ES EXISTENCIA

Primera válida → terminar

Puede haber armas sin usar.
No se exige maximizar poder.
No hay una capacidad de peso.

Hasta 3 guerreros · 12 armas

> “Superar” significa **>**. Una mochila de poder 10 no vence a un guerrero de poder 10.

Tipos, poder y requisitos.

Guerrero	Poder	Tipos
1	120	2
2	160	1, 3
3	80	3

> **P requiere Z.**
> Si P va a M2, Z también debe ir a M2.

La letra identifica el arma.
Su posición es el índice en el arreglo.

Arma	Poder	Tipo	Requisitos
Z	60	3	—
P	80	1	Z
R	38	2	—
D	25	2	R
E	49	2	—
F	57	1	—
G	68	3	—
H	35	2	Z, E
I	62	2	R
J	42	2	—
K	36	1	Z
L	54	3	—

El requisito tiene el mismo destino.

Seleccionar un requisito en otra mochila no cumple la regla.

P → Z

P en M2 obliga a Z en M2.

D, I → R

Cada arma seleccionada debe cumplir sus requisitos.

H → Z, E

H es tipo 2.
Z es tipo 3.

H no puede usarse con estos datos: ningún guerrero admite a la vez tipos 2 y 3. Esa conclusión depende de la entrada; no prohíbas H por su nombre en el código.

También K requiere Z. La asignación debe satisfacer todas las dependencias de las armas elegidas.

Validar todas las reglas.

```
sumas de poder = 0
para cada arma:
    si destino == 0: continuar
    si tipo no permitido: rechazar
    para cada requisito:
        si destino[requisito] != destino[arma]:
            rechazar
    acumular poder en su mochila

para cada guerrero:
    si suma <= poder del guerrero: rechazar
aceptar
```

La representación garantiza exclusividad.

Un arma tiene un solo destino.

El validador comprueba el resto.

Tipos, requisitos y poder estricto.

Las sumas se reinician para cada candidato.

Rompe una solución. Explica qué falló.

Cambia los destinos. El simulador comprueba esta asignación; el recorrido exhaustivo está en C++.

0 = sin usar

Z 60 · tipo 3 Requiere: — Mochila 2	**P** 80 · tipo 1 Requiere: Z Mochila 2	**R** 38 · tipo 2 Requiere: — Mochila 1	**D** 25 · tipo 2 Requiere: R Mochila 1	**E** 49 · tipo 2 Requiere: — Sin usar	**F** 57 · tipo 1 Requiere: — Mochila 2
G 68 · tipo 3 Requiere: — Mochila 3	**H** 35 · tipo 2 Requiere: Z, E Sin usar	**I** 62 · tipo 2 Requiere: R Mochila 1	**J** 42 · tipo 2 Requiere: — Sin usar	**K** 36 · tipo 1 Requiere: Z Sin usar	**L** 54 · tipo 3 Requiere: — Mochila 3

M1 · R, D, I Poder 125 > 120 · admite 2	**M2 · Z, P, F** Poder 197 > 160 · admite 1, 3	**M3 · G, L** Poder 122 > 80 · admite 3

Asignación válida. Tipos, requisitos en la misma mochila y poder estricto: correctos. Cada arma tiene un único destino.

El contador encuentra la primera válida.

```
base = cantidad_guerreros + 1;
int total = 1;
for (int i = 0; i < A; i++) total *= base;

for (int codigo = 0; codigo < total; codigo++) {
    cargarCromosoma(cromosoma, A, codigo, base);
    if (esAsignacionValida(/* datos */)) {
        copiar respuesta;
        return true;
    }
}
return false;
```

CASO MÁXIMO DEL ENUNCIADO

4^{12}

16 777 216

asignaciones posibles

> Es un esquema. En C++, ^ es XOR, no potencia. El archivo cuenta con multiplicación entera.

Implementación completa: po.cpp · Enumera asignaciones completas con ciclos, sin recursión.

Otra selección también puede ser correcta.

Primera solución de `po.cpp` para `datos1.txt`.

Mochila	Armas	Poder	Requisitos satisfechos
M1	R, I, J	**142 > 120**	I tiene a R.
M2	Z, P, K	**176 > 160**	P y K tienen a Z.
M3	G, L	**122 > 80**	No tienen requisitos.

CANDIDATOS PROBADOS HASTA ENCONTRARLA

10 751 068

Se comprueba el enunciado.

No se exige copiar la selección del ejemplo. El orden de enumeración puede cambiar la primera respuesta.

Prueba una regla a la vez.

Todos estos casos son pequeños: el razonamiento cabe en una línea.

Caso	Resultado	Regla que comprueba
Poder 10 contra guerrero de 10	No vence	La igualdad no basta.
A=6 requiere B=5; ambos permitidos	11 vence a 10	El requisito va junto.
Poder 20, tipo no permitido	No vence	Poder suficiente no basta.
Una arma para dos guerreros	No hay solución	No se reutiliza el arma.

> Los tres primeros casos están en `Codigo C++/casos`. Predice cada resultado antes de ejecutarlo.

Dos guerreros.
Tres armas.
¿Cuántos candidatos?

Interpreta `2 0 1`. Después explica cuándo puede detenerse el buscador.

Base 3 → **3^3 = 27 candidatos.**
A va a M2; B no se usa; C va a M1.
Po termina en la primera válida. La mochila de máximo peso debe comparar o alcanzar una cota superior demostrada.

Representar → Enumerar → Validar

Práctica, datos y C++ en esta carpeta. Fuente de Po: PUCP, LAB1 2024-1, pregunta 1.
Material de apoyo para estudio; no es una solución oficial de la universidad.